



MACHINE LEARNING

Alonso Monasterio



28 DE MARZO DE 2024

Contenido

1. Introducción al objetivo del problema	3
2. Pasos previos	3
3. Regresión logística y Red Neuronal	4
4. SelectKBest	6
5. Comparación modelos	7
6. Árbol de clasificación.....	8
7. Bagging y Random Forest.....	11
Bagging	11
Random Forest	12
8. Gradient Bosting y XGBoost	13
Gradient Boosting.....	13
XGBoost	13
9. SVM	14
10. Ensamblado de Bagging	15
11. Stacking	15

1. Introducción al objetivo del problema

El trabajo tiene como objetivo predecir una variable binaria mediante diferentes algoritmos de clasificación estudiados, a partir de un conjunto de datos que contiene información sobre características de personas como el sexo, edad, peso, altura... se pretende predecir si el individuo padece o no obesidad.

2. Pasos previos

Antes de poder realizar cualquier modelo, necesitamos realizar una depuración de las variables para mejorar la calidad de los datos. Para ello:

- 1- Análisis de valores perdidos: de nuestra muestra de 1.000 instancias se va a comprobar cuantos valores perdidos hay y, en función de ello ver cómo manejarlo.

```
In [7]: print(data.isnull().sum())
Gender                28
Age                   21
Height                20
Weight                20
family_history_with_overweight 16
FAVC                  26
FCVC                  26
NCP                   20
CAEC                  20
SMOKE                 18
CH20                  19
SCC                   22
FAF                   26
TUE                   19
CALC                  22
MTRANS                18
NObesidad             20
dtype: int64
```

Como podemos observar, la cantidad de datos faltantes es mínima por lo que se procede a eliminar los datos faltantes acabando con un conjunto de 929 instancias.

- 2- Variables categóricas: para poder realizar modelos, se necesitan transformar en numéricas binarias.

- a. Binarias: mediante función de transformación simple

```
#Las categorías binarias transformar a numéricas
data['Gender'] = data['Gender'].apply(lambda x: 0 if x=='Male' else 1)
data['family_history_with_overweight'] = data['family_history_with_overweight'].apply(lambda x: 0 if x=='yes' else 1)
data['FAVC'] = data['FAVC'].apply(lambda x: 0 if x=='yes' else 1)
data['SMOKE'] = data['SMOKE'].apply(lambda x: 0 if x=='yes' else 1)
data['SCC'] = data['SCC'].apply(lambda x: 0 if x=='yes' else 1)
```

- b. No binarias: aplicando codificación One Hot se obtiene un conjunto de datos nuevos con 23 columnas:

```
variables_cat_nobin = ['CAEC', 'CALC', 'MTRANS']

# Aplicar codificación one-hot a las variables categóricas
data2 = pd.get_dummies(data, columns=variables_cat_nobin, drop_first=True, dtype=float)
```

Gender	int64	CAEC_Frequently	float64
Age	float64	CAEC_Sometimes	float64
Height	float64	CAEC_no	float64
Weight	float64	CALC_Sometimes	float64
family_history_with_overweight	int64	CALC_no	float64
FAVC	int64	MTRANS_Bike	float64
FCVC	float64	MTRANS_Motorbike	float64
NCP	float64	MTRANS_Public_Transportation	float64
SMOKE	int64	MTRANS_Walking	float64
CH2O	float64		
SCC	int64		
FAF	float64		
TUE	float64		
NObeyesdad	object		

- 3- Variables numéricas: se procede a realizar una estandarización de las mismas para que todas las variables estén dentro de la misma escala, se eliminen sesgos...

```
scaler = StandardScaler()
data2[variables_numericas] = scaler.fit_transform(data2[variables_numericas])
```

- 4- Transformar la variable objetivo en binaria: para poder realizar clasificaciones, transformamos la variable objetivo donde 1 indica la presencia de cualquier tipo de obesidad y 0 que no tiene obesidad y se comprueba que las dos clases estén representadas equitativamente.

```
data2['NObeyesdad'] = data2['NObeyesdad'].apply(lambda x: 1 if 'Obesity' in str(x) else 0)
```

```
NObeyesdad
0      490
1      439
Name: count, dtype: int64
```

Con este nuevo conjunto de datos se van a realizar los modelos solicitados.

```
# Separando las variables independientes y la variable objetivo
X = data2.drop('NObeyesdad', axis=1)
y = data2['NObeyesdad']

# Dividiendo los datos en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=seed, stratify=y)
```

3. Regresión logística y Red Neuronal

Para obtener el mejor modelo de regresión logística vamos a realizar una búsqueda de hiperparámetros mediante GridSearchCV.

```
# creando y entrenando el modelo de regresión logística
modelo_reg_log = LogisticRegression(max_iter=100, random_state=seed)
param_grid = {
    'C': [0.01, 0.005, 0.02, 0.03],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear']
}

search = GridSearchCV(modelo_reg_log, param_grid, cv=5, scoring='accuracy', verbose=1, n_jobs=-1)
pipeline = Pipeline(steps=[('busqueda', search)])

pipeline.fit(X, y)
```

Los mejores hiperparámetros obtenidos y su puntuación son:

```
Mejores parámetros: {'C': 0.03, 'penalty': 'l2', 'solver': 'liblinear'}  
Mejor score (Accuracy): 0.9580412670735251
```

Estos hiperparámetros nos indican:

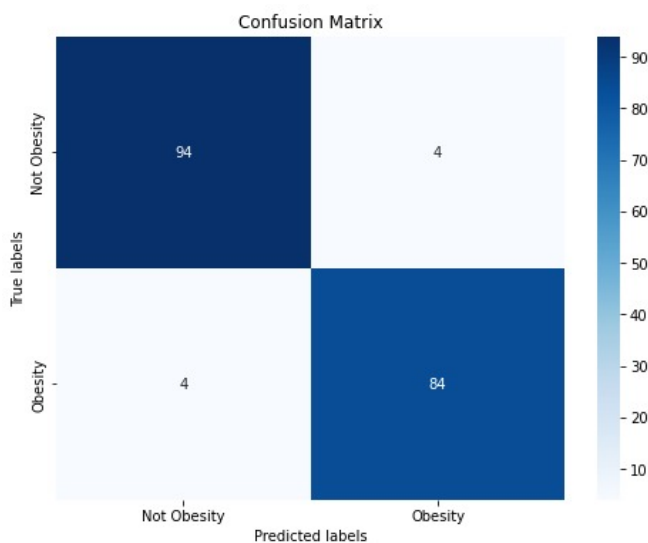
- C: indica la fuerza de regularización para un modelo ayudando a prevenir sobreajuste, para ello se ha escogido un valor relativamente bajo como 0.03
- L2: ayuda a reducir el sobreajuste penalizando el cuadrado de los coeficientes
- Liblinear: se ha elegido este solucionador debido a la cantidad de datos del conjunto a estudiar.

Vamos a probar el modelo obtenido con los datos en test:

```
modelo_optimizado = LogisticRegression(C=0.03, penalty='l2', solver='liblinear', max_iter=100, random_state=seed)  
modelo_optimizado.fit(X_train, y_train)  
  
y_pred_optimizado = modelo_optimizado.predict(X_test)
```

```
In [15]: print(f"Accuracy del modelo optimizado: {accuracy_optimizado}")  
Accuracy del modelo optimizado: 0.956989247311828
```

Con estos resultados se concluye que el modelo clasifica correctamente la gran mayoría de datos y al existir muy poca diferencia entre train y test se concluye que generaliza bien nuevos datos, esto se puede observar con la matriz de confusión del modelo para ver la cantidad de falsos positivos y falsos negativos ha clasificado:



Una vez obtenido el mejor modelo se va a buscar la mejor red neuronal en términos de accuracy, repitiendo el proceso de GridSearchCV pero con un conjunto diferente de hiperparámetros a probar.

```

param_grid_nn = {
    'hidden_layer_sizes': [(2,), (20,), (50,)],
    'activation': ['tanh', 'relu'],
    'solver': ['sgd', 'adam'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate_init': [0.001, 0.01]
}

mlp = MLPClassifier(max_iter=100, random_state=seed)

grid_search_nn = GridSearchCV(
    mlp, param_grid_nn, cv=5, scoring='accuracy', verbose=1, n_jobs=-1)

# Ajuste del modelo con GridSearchCV
grid_search_nn.fit(X_train, y_train)

```

El mejor modelo obtenido ha sido:

```

.... print( 'Mejor accuracy: ', grid_search_nn.best_score_)
Mejores hiperparámetros: {'activation': 'tanh', 'alpha': 0.01, 'hidden_layer_sizes': (20,),
'learning_rate_init': 0.01, 'solver': 'adam'}
Mejor accuracy: 0.9959731543624162

```

Con una función de activación tangente, Alpha = 0.01 intentando prevenir sobreajuste, se ha elegido una sola capa oculta con 20 neuronas y una tasa de aprendizaje media.

La precisión tan alta en entrenamiento puede indicar que el modelo esta sobreajustado.

La precisión obtenida en test es de

```

In [54]: print(f"Accuracy del modelo optimizado: {accuracy_nn}")
Accuracy del modelo optimizado: 0.9946236559139785

```

4. SelectKBest

Aplicamos KBest al conjunto de datos

```

from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(score_func=f_classif, k=4)

X_new = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

# Obtener los índices de las características seleccionadas
index_seleccionadas = selector.get_support(indices=True)
selected_names = X_train.columns[index_seleccionadas]
print("Características seleccionadas:", selected_names)

```

Las variables seleccionadas son:

```

In [63]: print("Características seleccionadas:", selected_names)
Características seleccionadas: Index(['Weight', 'family_history_with_overweight', 'FAVC',
'CAEC_Sometimes'], dtype='object')

```

Una vez obtenidas las variables con mayor importancia, se procede con la búsqueda de la mejor red neuronal en términos de AUC:


```

param_grid_nn = {
    'hidden_layer_sizes': [(10,),(20,),(50,)],
    'activation': ['tanh', 'relu'],
    'solver': ['sgd', 'adam'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate_init': [0.001, 0.01, 0.1],
}

# Inicializar el clasificador de red neuronal
mlp = MLPClassifier(max_iter=100, random_state=seed)

# Configurar GridSearchCV para buscar la mejor red neuronal basada en AUC
grid_search_nn = GridSearchCV(mlp, param_grid_nn, n_jobs=-1, cv=5, scoring='roc_auc', verbose=1)

# Ajustar GridSearchCV en el conjunto de entrenamiento usando solo las características seleccionadas
grid_search_nn.fit(X_train_new, y_train)

```

Obteniendo el siguiente modelo:

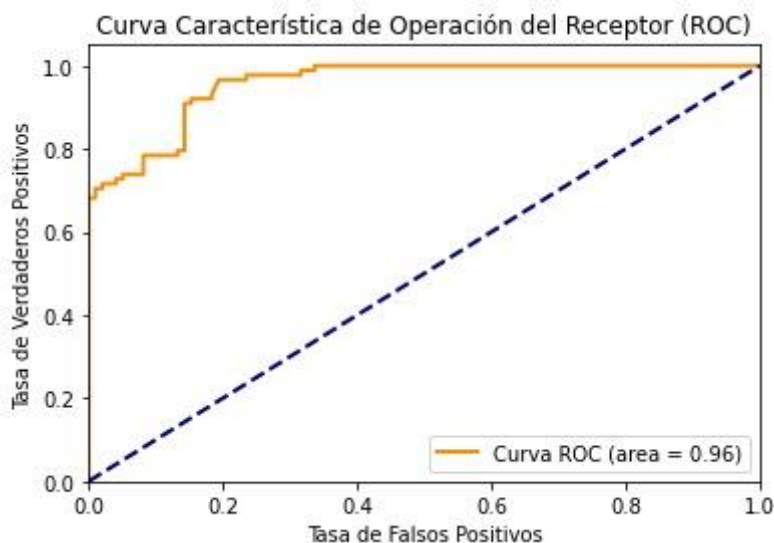
```

... print('AUC ROC en Test: {roc_auc}')
Mejor Red Neuronal: MLPClassifier(activation='tanh', alpha=0.01, hidden_layer_sizes=(20,),
    learning_rate_init=0.1, max_iter=100, random_state=4009)
Parámetros: {'activation': 'tanh', 'alpha': 0.01, 'hidden_layer_sizes': (20,), 'learning_rate_init': 0.1, 'solver': 'adam'}
AUC ROC en Test: 0.9566906307977736

```

Del modelo concluimos que tiene una regularización moderada de L2, una sola capa oculta con 20 neuronas debido a la simplicidad de los datos y el uso del optimizador 'adam' para ajustar automáticamente la tasa de aprendizaje.

La curva del modelo obtenido es:



Tiene una gran capacidad de clasificar con precisión así como una alta confianza en las predicciones que realiza.

5. Comparación modelos

Se puede observar como los dos modelos obtenidos de los dos apartados son buenos modelos aunque el primero podría sufrir de sobreajuste, a su vez el segundo modelo aún habiendo eliminado la gran mayoría de variables obtiene un área de la curva ROC muy buena por lo que en términos prácticos el segundo sería mejor modelo.

6. Árbol de clasificación

Para encontrar el mejor árbol se va a utilizar otra vez GridSearchCV y se definen los 4 métodos de validación de bondad

```
modelo_arbol = DecisionTreeClassifier(random_state=seed)
parametros_arbol = {
    'max_depth': [2,5],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy'],
    'max_features': ['auto', 'sqrt', 'log2', None]
}

scoring = {
    'accuracy': make_scorer(accuracy_score),
    'roc_auc': 'roc_auc',
    'precision': make_scorer(precision_score, zero_division=0),
    'recall': make_scorer(recall_score)
}

busqueda_arbol = GridSearchCV(modelo_arbol, parametros_arbol, cv=5, scoring=scoring, refit='roc_auc', verbose=1, n_jobs=-1)
busqueda_arbol.fit(X_train, y_train)
```

El mejor árbol obtenido es

```
In [103]: print("Parámetros: {mejores_parametros}")
Parámetros: {'criterion': 'entropy', 'max_depth': 5, 'max_features': None, 'min_samples_leaf': 1,
'min_samples_split': 2}
```

Con este conjunto de hiperparámetros, se da un modelo que busca encontrar divisiones que maximicen ganar información sin ser un árbol muy profundo para intentar prevenir el sobreajuste.

Con las siguientes medidas de bondad que indica que el árbol obtenido es un árbol muy efectivo a la hora de clasificar con un equilibrio a la hora de clasificar falsos positivos y falsos negativos.

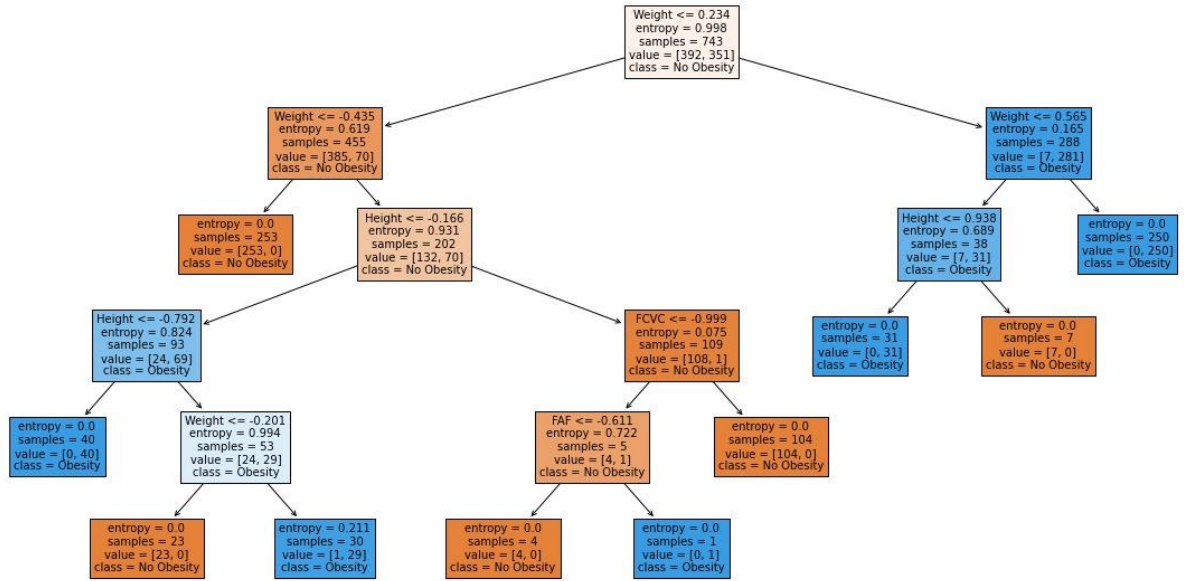
```
In [104]: print(f"Precisión en Test: {best_accuracy}")
Precisión en Test: 0.978494623655914

In [105]: print(f"AUC ROC en Test: {best_roc_auc}")
AUC ROC en Test: 0.9773886827458256

In [106]: print(f"Precisión en Test: {best_precision}")
Precisión en Test: 0.9883720930232558

In [107]: print(f"Recall en Test: {best_recall}")
Recall en Test: 0.9659090909090909
```

EL árbol obtenido gráficamente es



Las reglas son:

--- Weight <= 0.23

| |--- Weight <= -0.43

| | |--- class: 0

| |--- Weight > -0.43

| | |--- Height <= -0.17

| | | |--- Height <= -0.79

| | | | |--- class: 1

| | | |--- Height > -0.79

| | | |--- Weight <= -0.20

| | | | |--- class: 0

| | | |--- Weight > -0.20

| | | | |--- class: 1

| | |--- Height > -0.17

| | | |--- FCVC <= -1.00

| | | | |--- FAF <= -0.61

| | | | |--- class: 0

| | | | |--- FAF > -0.61

| | | | |--- class: 1

| | | |--- FCVC > -1.00

```

| | | | |--- class: 0
|--- Weight > 0.23
| |--- Weight <= 0.57
| | |--- Height <= 0.94
| | | |--- class: 1
| | |--- Height > 0.94
| | | |--- class: 0
| |--- Weight > 0.57
| | |--- class: 1

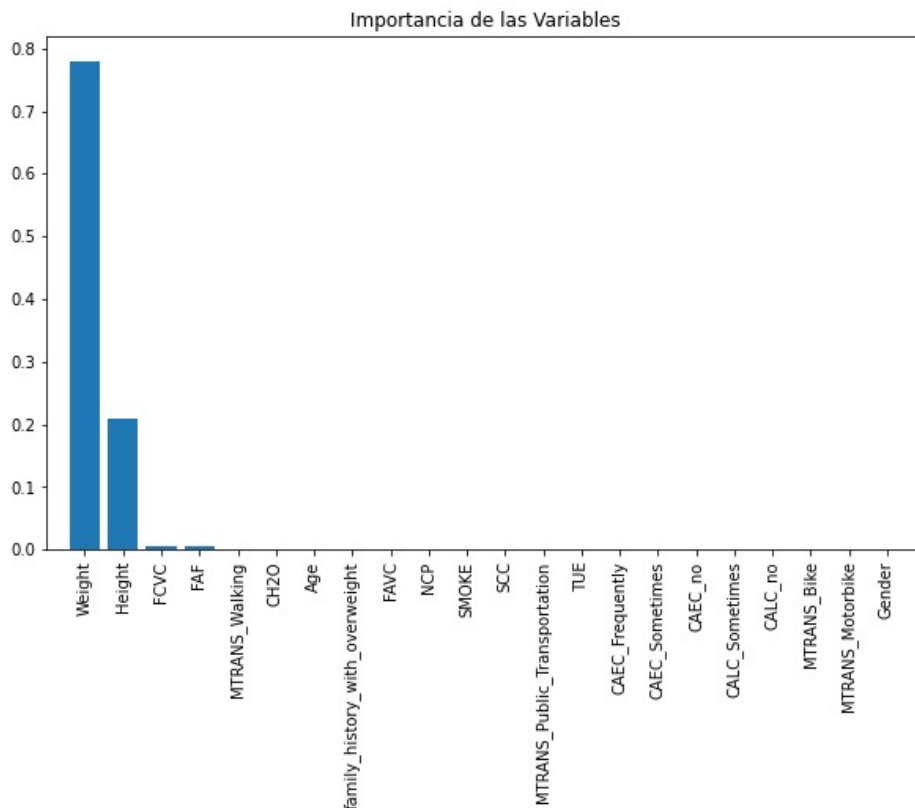
```

Las decisiones del árbol obtenido son:

- En el nivel más alto, el árbol primero pregunta si el Weight es menor o igual a 0.23 (un valor estandarizado). Esta división sugiere que el peso es la característica más importante para la clasificación.
 - o Para aquellos con un Weight menor o igual a 0.23, el árbol luego pregunta si el Weight es menor o igual a -0.43.
 - Si el peso es menor o igual a -0.43, el árbol clasifica la instancia como clase 0 (no obesidad).
 - Si el peso es mayor a -0.43, el árbol considera la altura.
 - Si la Height es menor o igual a -0.17, hay una nueva subdivisión basada en la altura.
 - o Si la Height es menor o igual a -0.79, la instancia se clasifica como clase 1 (obesidad).
 - o Si la Height es mayor a -0.79, el árbol considera nuevamente el peso.
 - Si el Weight es menor o igual a -0.20, clasifica la instancia como clase 0.
 - Si el Weight es mayor a -0.20, clasifica la instancia como clase 1.
 - o Si la Height es mayor a -0.17, el árbol pregunta por la FCVC.
 - Si la FCVC es menor o igual a -1.00, se considera la frecuencia de actividad física (FAF).
 - Si la FAF es menor o igual a -0.61, la instancia se clasifica como clase 0.
 - Si la FAF es mayor a -0.61, la instancia se clasifica como clase 1.
 - Si la FCVC es mayor a -1.00, la instancia se clasifica como clase 0.
- Para aquellos con un Weight mayor a 0.23, el árbol hace otra serie de preguntas sobre el peso.
 - o Si el Weight es menor o igual a 0.57, entonces considera la Height.

- Si la Height es menor o igual a 0.94, la instancia se clasifica como clase 1.
- Si la Height es mayor a 0.94, la instancia se clasifica como clase 0.
- Si el Weight es mayor a 0.57, la instancia se clasifica directamente como clase 1.

El árbol muestra una clara dependencia en el peso, lo cual tiene sentido a la hora de predecir la obesidad en un individuo. La altura juega también un papel importante y los hábitos alimenticios y actividad física sirven para ajustes adicionales en ciertos nodos.



7. Bagging y Random Forest

Para encontrar el mejor modelo de Bagging y Random Forest se procede como hasta ahora con GridSearchCV.

Bagging

```
base_estimator = DecisionTreeClassifier(random_state=seed)

parametros_bagging = {
    'base_estimator__max_depth': [10, 20, 50, 75, 100],
    'n_estimators': [10, 50, 100, 200], # Número de árboles
    'max_samples': [0.5, 0.7, 1.0], # Máximo número de muestras por árbol
    'max_features': [0.5, 0.7, 1.0],
    'base_estimator__criterion': ['gini', 'entropy']
}

bagging_grid_search = GridSearchCV(BaggingClassifier(estimator=base_estimator, random_state=seed),
                                   parametros_bagging, cv=5, scoring='accuracy', n_jobs=-1)

bagging_grid_search.fit(X_train, y_train)
```

Se obtiene el siguiente modelo, con una alta precisión tanto en train como en test. La entropía como medida de los nodos con la profundidad de 20 captura la complejidad de los datos siendo un modelo robusto.

```
Mejores parámetros (Bagging): {'base_estimator__criterion': 'entropy', 'base_estimator__max_depth': 20, 'max_features': 0.7, 'max_samples': 1.0, 'n_estimators': 100}
Mejor puntuación (Bagging): 0.9892526754942862
Precisión en Test: 0.978494623655914
```

Random Forest

Igual que los anteriores modelos se utiliza GridSearchCV para realizar la búsqueda paramétrica

```
parametros_rf = {
    'max_depth': [10, 20, 30, None],
    'n_estimators': [10, 50, 100, 200],
    'max_features': ['auto', 'sqrt', 'log2'],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

rf = RandomForestClassifier(random_state=seed)
rf_grid_search = GridSearchCV(estimator=rf, param_grid=parametros_rf, cv=5, scoring='accuracy', n_jobs=-1)

rf_grid_search.fit(X_train, y_train)
```

Obteniendo el siguiente modelo:

```
... print('Precisión en Test: {accuracy_test}')
Mejores parámetros (Random Forest): {'bootstrap': False, 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 200}
Mejor puntuación (Random Forest): 0.9865590422637404
Precisión en Test: 0.9516129032258065
```

- Bootstrap: False sugiere que el tamaño del conjunto de datos es suficiente para permitir que cada árbol del bosque se entrene con todos los datos disponibles.
- Max Depth: profundidad máxima de 10 niveles en cada árbol para prevenir sobreajuste
- Max Features: 'sqrt' (cada árbol utiliza la raíz cuadrada del número total de características para determinar la mejor división).
- Min Samples Leaf: 1.
- Min Samples Split: 5.
- N Estimators: 200 (el número de árboles en el bosque).

La precisión del modelo para train y para test son altas sin haber mucha diferencia entre ellas lo que da en un modelo equilibrado con alta precisión en sus clasificaciones.

8. Gradient Bosting y XGBoost

Gradient Boosting

```
parametros_gb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 4, 5],
    'min_samples_split': [2, 3],
    'min_samples_leaf': [1, 2],
    'max_features': ['sqrt', 'log2', None]
}

gb_clf = GradientBoostingClassifier(random_state=seed)

grid_search_gb = GridSearchCV(gb_clf, parametros_gb, cv=5, scoring='accuracy', n_jobs=-1)

grid_search_gb.fit(X_train, y_train)
```

Los parámetros obtenidos para el modelo de Gradient Boosting son:

```
print('Precisión en Test: {accuracy_test}')
Mejores parámetros (Gradient Boosting): {'learning_rate': 0.1, 'max_depth': 4, 'max_features': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 300}
Mejor puntuación (Gradient Boosting): 0.9959640848902593
Precisión en Test: 0.978494623655914
```

- Tasa de Aprendizaje (learning_rate): 0.1, permite un equilibrio entre la velocidad de entrenamiento y el riesgo de no converger a una solución óptima.
- Número de Estimadores (n_estimators): 300, que el modelo utiliza 300 árboles sucesivos para corregir los errores de los árboles anteriores.
- Profundidad Máxima (max_depth): 4 para evitar en la medida de lo posible sobreajuste
- Número Máximo de Características (max_features): None, todas las características se consideran al buscar la mejor división.
- Mínimo de Muestras por Hoja (min_samples_leaf): 1
- Mínimo de Muestras para Dividir (min_samples_split): 2

El modelo obtiene una alta precisión en el conjunto de train lo que puede dar a pensar que el modelo puede padecer de sobreajuste pero un 97,85% en test sugiriendo que el modelo tiene una excelente capacidad de generalización.

En resumen, este modelo de Gradient Boosting parece estar bien ajustado con un buen rendimiento generalizado, y los hiperparámetros elegidos apuntan a un modelo que ha encontrado un equilibrio efectivo entre sesgo y varianza.

XGBoost

Aplicando GridSearchCV para realizar la búsqueda paramétrica para XGBoost

```
parametros_xgb = {
    'n_estimators': [10, 20, 30, 50],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 4, 5],
    'min_child_weight': [1, 2, 5],
    'gamma': [0, 0.1, 0.2],
    'subsample': [0.8, 1],
    'colsample_bytree': [0.8, 1]
}

xgb_clf = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=seed)

grid_search_xgb = GridSearchCV(xgb_clf, parametros_xgb, cv=5, scoring='accuracy', n_jobs=-1)

grid_search_xgb.fit(X_train, y_train)
```

Los parámetros obtenidos son:

- `colsample_bytree`: 1, lo que significa que se utiliza la totalidad de las características al construir cada árbol.
- `gamma`: 0, indicando que no hay regularización adicional basada en el parámetro `gamma`.
- `learning_rate`: 0.2, que el modelo es capaz de aprender rápidamente.
- `max_depth`: 3, para prevenir el sobreajuste y aumentando la velocidad de entrenamiento.
- `min_child_weight`: 1.
- `n_estimators`: 50, el número de árboles utilizados en el modelo, que es un balance entre rendimiento y velocidad de entrenamiento.
- `subsample`: 1, lo que implica que se utiliza todo el conjunto de datos para construir los árboles, sin submuestreo.

```
Mejores parámetros (XGBoost): {'colsample_bytree': 1, 'gamma': 0, 'learning_rate': 0.2, 'max_depth': 3, 'min_child_weight': 1, 'n_estimators': 50, 'subsample': 1}
Mejor puntuación (XGBoost): 0.9932795211318701
Precisión en Test: 0.989247311827957
```

El modelo ha obtenido una alta precisión en el conjunto train indicando un sobreajuste del modelo pero ha obtenido también un alto porcentaje en test por lo que se puede concluir que el modelo obtenido es preciso y esta ajustado de forma correcta.

9. SVM

Se aplica GridSearchCV para los kernels 'rbf' y 'linear'

```
parametros_svm = {
    'kernel': ['rbf', 'linear'],
    'C': [0.1, 1, 10],
    'gamma': ['scale', 'auto'],
}

svm_clf = SVC(random_state=seed)

grid_search_svm = GridSearchCV(svm_clf, parametros_svm, cv=5, scoring='accuracy', n_jobs=-1)

grid_search_svm.fit(X_train, y_train)
```

El mejor modelo obtenido tiene los siguientes parámetros:

- Kernel: RBF, efectivo para manejar clasificaciones no lineales
- C: 10, compromiso entre un margen suave y clasificación correcta para evitar sobreajustes.
- Gamma: Scale para estabilizar los datos.

```
.... print() Precisión en Test: {accuracy_test}
Mejores parámetros (SVM): {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
Mejor puntuación (SVM): 0.9932795211318701
Precisión en Test: 0.989247311827957
```


La alta predicción en el conjunto de entrenamiento puede indicar sobreajuste pero al comparar con la precisión en test que también es alta se puede concluir en que el modelo clasifica correctamente la mayoría de los datos aunque este podría estar sobreajustado.

10. Ensamblado de Bagging

Para realizar el ensamblado de Bagging se ha implementado el modelo de regresión logística obtenido en el apartado 1.

```
base_clf = LogisticRegression(C=0.03, penalty='l2', solver='liblinear', max_iter=100, random_state=seed)
bagging_clf = BaggingClassifier(base_estimator=base_clf, n_estimators=10, random_state=seed, n_jobs=-1)
bagging_clf.fit(X_train, y_train)
accuracy = bagging_clf.score(X_test, y_test)
```

El modelo obtenido consigue una precisión del 94,6% lo que indica que el modelo tiene un alto rendimiento para clasificar nuevas instancias.

```
[accuracy]
Accuracy del modelo de Bagging con Regresión Logística como clasificador base: 0.946236559139785
```

11. Stacking

El Stacking es un método de ensamblaje que utiliza las predicciones de varios modelos base como entrada para un modelo de nivel superior que realiza la predicción final.

Para la selección de modelos base se han elegido: Árbol de decisión, SVM y una regresión logística los tres con sus parámetros por defecto.

El modelo de ensamblaje ha sido el modelo de Regresión Logística obtenido en el primer apartado.

```
modelos_base = [
    ('dt_clf', DecisionTreeClassifier(random_state=seed)),
    ('svm_clf', SVC(probability=True, random_state=seed)),
    ('lr_clf', LogisticRegression(random_state=seed))
]

meta_modelo = LogisticRegression(C=0.03, penalty='l2', solver='liblinear', max_iter=100, random_state=seed)
stacking_clf = StackingClassifier(estimators=modelos_base, final_estimator=meta_modelo, cv=5, n_jobs=-1)
stacking_clf.fit(X_train, y_train)
accuracy = stacking_clf.score(X_test, y_test)
```

El modelo obtenido da una precisión en test del 97,31% indicando que este modelo combina de manera eficaz los modelos base que se le han enviado, aun así la precisión tan alta del modelo puede indicar una leve tendencia de sobreajuste.

```
In [175]: print(f'Accuracy del modelo de Stacking: {accuracy:.4f}')
Accuracy del modelo de Stacking: 0.9731
```